

Project Report: JavaLibrary

Equipe

- **Marcos Vinicius Reballo** - Nº USP: 7576746
 - **Arthur Gagliardi Azorli** - Nº USP: 16855452
 - **Pedro Kemp** - Nº USP: 17064431
-

1. Como Rodar e Breve Descrição

A **JavaLibrary** é uma aplicação desktop completa de gerenciamento de biblioteca, construída visando atender de forma rigorosa as especificações do módulo "Compact Version" da disciplina SCC0504. O projeto foi integralmente desenvolvido e faturado em linguagem Java pura e utiliza o framework **JavaFX** moderno para a construção intuitiva das interfaces de usuário (GUI).

O sistema oferece uma série de controles, limitando restrições essenciais para três entidades chaves:

1. **Livros (Books):** Processo de adição da catalogação através de Identificadores (ISBN), títulos e controle de estoque de exemplares disponíveis (cópias ativas).
2. **Usuários (Patrons):** Cadastro e exclusão de clientes da biblioteca. Um sistema de integridade garante que usuários que possuam livros pendentes em aberto não podem ser deletados.
3. **Empréstimos (Loans):** O ciclo de vida do negócio ocorre aqui. A aplicação calcula o prazo regular (devolução padrão estipulada em 14 dias), bloqueia o empréstimo imediato se as cópias do livro já estiverem esgotadas, decrementa a quantia de estoques e incrementa na devolução.

A aplicação exclui o uso de bancos de dados relacionais e em seu lugar opta pela abordagem de I/O de Arquivos (Persistência em Disco), manipulando arquivos independentes formatados em **JSON** (`books.json`, `users.json`, `loans.json`) estocados no mesmo diretório local de inicialização.

Instruções de Execução (How to run it)

Para que a compilação do pacote, as variáveis e o lançamento sejam corretos, siga a instrução:

1. Confirme as variáveis locais do **Java Development Kit (JDK) 21** e da ferramenta **Maven** na sua máquina.
2. Inicie uma instância de Terminal ou Prompt de Comando.
3. Acesse a pasta central do projeto (que contempla a localização do pacote `pom.xml`).

4. Remova compilações indesejadas anteriores (`clean`), compile e levante o aplicativo com a task unificada JavaFX abaixo:

```
mvn clean javafx:run
```

5. O sistema invocará uma nova janela visual interativa, pronta para uso sem a necessidade de instâncias de terminal em background.

2. Conceitos POO aplicados e Justificativas

Como essência primária da disciplina avaliativa, todos os comportamentos lógicos e estruturais foram espalhados baseando-se nos paradigmas da Programação Orientada a Objetos:

- **Encapsulamento (Encapsulation e Data Hiding):** Evidente nos arquivos do pacote `domains` (como `Book`, `User`, `Loan`). A blindagem de campos confidenciais foi garantida pela declaração de atributos como estritamente privados (ex: `private int copiesCount`). Esse ocultamento protege contra a exposição crua, forçando que mudanças no objeto e sua leitura partam somente dos métodos assessores reguladores (getters e setters) e lógicas interativas (`isFinished()`), impossibilitando efeitos colaterais por entidades alheias tentando modificar dados arbitrariamente.
 - **Herança (Inheritance):** Usufruímos profundamente deste conceito para mapear o comportamento da classe nativa pai (base) e adaptar para nosso uso. O reflexo claro desta aplicação é a `BookNotAvailableException`, na qual estendemos as engrenagens embutidas da classe raiz base nativa `Exception` do Java (usando a assinatura `extends`). A herança concede a nós a autonomia de capturarmos o erro da indisponibilidade de livro sem confundir com falhas de I/O em blocos de `try-catch`, tratando de modo limpo e focado no domínio.
 - **Polimorfismo (Polymorphism):** Uma das características mais essenciais que foram sobreescritas partindo do ancestral principal de classe (a classe `Object`). Modelos de domínio, como `User` e `Book`, subscreveram a implementação nativa usando os decoradores de `@Override` em cima dos métodos `equals()` e `hashCode()`. A intenção polimorfa aqui alterou o padrão estrutural de como e quando dois objetos instanciados da memória são tratados como sendo "o mesmo" (se igualando pelos atributos ID ou ISBN). Isso nos propiciou organizar listagens, pesquisas e validações nas coleções dinâmicas sem duplicatas.
 - **Modelação Estrutural Singleton:** Invocamos um acoplamento protetivo para o design isolando classes com repositório num bloco `Singleton` (`BookRepository`, `UserRepository`, `JsonService`). Elas privatizam sua própria construção e devolvem apenas a mesma e estática instância em memória durante toda a vida da aplicação (`.getInstance()`), blindando a estabilidade de gravação no arquivo com estado universal único.
-

3. Principais Desafios Encontrados e Soluções

Diversos desafios que testaram o isolamento das arquiteturas e da segurança das funções emergiram em momentos da implementação:

- **Desafio 1: Tabela Dinâmica entre Classes Desassociadas (Loan Overview)**

- **Problema Encontrado:** Ao gerar a tabela de Listagem (Overview) para cumprir uma das regras primárias, fomos deparados com um impasse em relação a classe pura `Loan` — que persiste unicamente os códigos puros UUID e numérico (ISBN), desrespeitando o layout de texto da Interface que pedia expressamente Nome Completo e Título de cada registro em formato humano legível.
- **Solução:** Em vez de quebrar a lógica do JSON anexando repetição de nomes em múltiplos lugares e inflando as classes, interceptamos a injeção em JavaFX. Nós recriamos um evento de ouvinte em `LoanController` usando interligações via `SimpleStringProperty` e `CellValueFactory`. A Interface agora traduz a célula dinamicamente, enviando o ID em tempo real nos Repositórios originais (`UserRepository` e `BookRepository`), extraindo o nome do objeto em um mapa virtual e exibindo ao usuário sem sujar a estrutura base de empréstimos.

- **Desafio 2: Falha Silenciosa de Integridade (Estoque de Livros)**

- **Problema Encontrado:** Apesar dos dados salvarem, percebemos que um empréstimo ocorria com um comportamento de "falha-silenciosa" (silent failure), diminuindo quantidades além de zero em estoques negativos de livro.
- **Solução:** Substituímos as lógicas fracas nas rotinas visuais que avaliavam superficialmente e deslocamos o bloqueio de segurança em cadeia pesada dentro do `.loanBook()`. Usando injeção em blocos transacionais base, o método verifica a existência exata do ponteiro `.getCopiesCount()`. Se detectado erro, a aplicação é instruída a frear, lançando nossa classe herdada `BookNotAvailableException`. A resposta visual capta a Exceção por um "throw" e reverte todo o ciclo.

- **Desafio 3: Peso Custoso de Escrita em Discos Nativos**

- **Problema Encontrado:** Originalmente planejado como entrada textual, o trabalho de conversão e castamento em tempo real (instanciando `HashMaps` linha a linha manualmente) traria sobrecarga de reescrita em disco e propensão a imprecisão humana de manipulação de código na conversão e deleção do texto (ex: lidar iterativamente para deletar uma simples vírgula).
- **Solução:** Pivotamos integralmente nossa estratégia de controle para Formatação Estruturada **JSON** utilizando `Reflection` na biblioteca popular **GSON**. Isolamos a repetição das tentativas em uma barreira `Generic Service` (tipagem de `Cast` explícito como `<T>`). Dessa forma, seja processando Livros (`<ArrayList<Book>>`) ou clientes únicos, as regras

I/O foram substituídas na íntegra por chamadas elegantes de uma linha (como `JsonService.getInstance().loadJson()`).

4. Plano de Testes (Test Plan)

A validação da robustez, integridade de dados e conformidade das regras de negócio foi executada através do desenvolvimento de uma suíte de testes automatizados utilizando o framework **JUnit 5 (Jupiter)**. Os testes cobrem desde componentes utilitários simples até a simulação transacional de estado na camada de persistência em disco.

4.1. Escopo e Casos de Testes Automatizados

- **CriptoTest (`CriptoTest.java`):**
 - Validação do algoritmo hash MD5 com uma entrada comum (`"123456"`).
 - Validação de hash para string vazia (`""`).
 - Garantia de que entradas distintas produzem hashes distintos.
- **UserTest (`UserTest.java`):**
 - Criação de usuário com IDs auto-gerados (`UUID.randomUUID()`) ou explícitos.
 - Teste do mecanismo de checagem case-insensitive para hashes de senhas (`isPasswordCorrect()`).
 - Validação da regra de igualdade estrutural baseada em dados de perfil (`equals()` e `hashCode()`).
- **BookTest (`BookTest.java`):**
 - Construção e mutação de estados do livro (controle incremental e decremental de cópias ativas).
 - Testes de correspondência de equivalência por ISBN.
- **LoanTest (`LoanTest.java`):**
 - Ciclo de vida básico e datas de previsão do empréstimo.
 - Lógica de detecção de empréstimo devolvido (`isFinished()`).
 - Lógica de empréstimo atrasado (`isDelayed()`) baseado na data limite `endPrevision` comparada com o horário do sistema.
- **RepositoryIntegrationTest (`RepositoryIntegrationTest.java`):**
 - Testes de integração fim a fim dos repositórios (`UserRepository`, `BookRepository` e `LoanRepository`).

- **Segurança no Ciclo de Teste:** Rotinas de `@BeforeAll` e `@AfterAll` realizam o backup completo dos dados de produção em arquivos temporários e os restauram após o ciclo de testes, impedindo corrupção no workspace.
- **Validação da regra de negócio crítica:** lançamento da exceção customizada `BookNotAvailableException` ao tentar emprestar livros cujo estoque de cópias ativas chegou a 0.

5. Resultados dos Testes (Test Results)

A execução da suíte de testes foi acionada com sucesso no ambiente local utilizando a ferramenta de build Maven. Todos os 16 casos de teste escritos foram executados, resultando em 100% de sucesso sem falhas ou erros.

5.1. Log de Saída do Maven (Surefire Reports)

```
-----  
T E S T S  
-----
```

```
Running br.edu.usp.javalibrary.javalibrary.service.domains.BookTest
```

```
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.002 sec
```

```
Running br.edu.usp.javalibrary.javalibrary.service.domains.LoanTest
```

```
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.138 sec
```

```
Running br.edu.usp.javalibrary.javalibrary.service.domains.UserTest
```

```
Tests run: 4, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.014 sec
```

```
Running br.edu.usp.javalibrary.javalibrary.service.repository.RepositoryIntegration
```

```
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.479 sec
```

```
Running br.edu.usp.javalibrary.javalibrary.service.utils.CriptoTest
```

```
Tests run: 3, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.012 sec
```

```
Results :
```

```
Tests run: 16, Failures: 0, Errors: 0, Skipped: 0
```

```
[INFO] -----
```

```
[INFO] BUILD SUCCESS
```

```
[INFO] -----
```

```
[INFO] Total time: 31.434 s
```

```
[INFO] Finished at: 2026-06-23T19:54:27-03:00
```

```
[INFO] -----
```